PROYECTO FINAL

Programacion paralela

Erick Perez Peloz

Demi Alexander Moore

Johán Manuel Vicente B.

Diego Marte

Elmer Montilla Castro

22 de abril de 2026

Indice

PRESENTACION 1
Introduccion 3
1.1 Presentacion general del proyecto 3
1.2 Justificacion 3
1.3 Objetivo general 4
2. Descripcion del Problema 5
2.1 Contexto del problema seleccionado 5
2.2 Aplicacion del problema en un escenario real 5
2.3 Importancia del paralelismo en la solucion 6
3. Cumplimiento de los Requisitos del Proyecto 7
4. Diseno de la Solucion 9
5. Implementacion Tecnica 11
6. Evaluacion de Desempeno 13
6.1 Analisis de cuellos de botella o limitaciones 14
7. Trabajo en Equipo 15
8. Conclusiones 16
9. Referencias 17
10. Anexos 18

Introduccion

1.1 Presentacion general del proyecto

RouteX GPS RD es un sistema de consola desarrollado en C# que simula una aplicacion GPS capaz de encontrar la mejor ruta entre dos puntos dentro de una red de 1,000 nodos interconectados. El sistema lanza cuatro algoritmos de busqueda simultaneamente en paralelo, cada uno con un criterio de optimizacion distinto, compitiendo especulativamente para hallar y reportar rutas validas al usuario.

El sistema es capaz de calcular y ofrecer al usuario:

- La ruta de menor costo total entre origen y destino (Dijkstra clasico).
- La ruta con menos paradas intermedias (BFS - Busqueda en anchura).
- La ruta que evita conexiones con costo elevado, simulando la evasion de peajes (Dijkstra Filtrado con umbral de 70).
- La ruta mas directa segun costo inmediato en cada paso (algoritmo Greedy).

Al combinar la descomposicion especulativa con el paralelismo del Task Parallel Library (TPL) de .NET, el sistema responde al usuario tan pronto como el algoritmo mas rapido encuentra una ruta valida, sin necesidad de esperar a que los demas terminen.

1.2 Justificacion

El problema de busqueda de rutas es uno de los casos de uso mas representativos del mundo real donde el paralelismo aporta un beneficio concreto y medible. Aplicaciones como Google Maps, Waze y sistemas de navegacion aerea ejecutan multiples algoritmos en paralelo para dar al usuario la mejor opcion en el menor tiempo posible.

La justificacion del uso del paralelismo en este proyecto radica en tres factores principales. Primero, la escala del problema: la matriz de costos de 1,000 por 1,000 nodos representa un millon de celdas, lo que hace que el procesamiento secuencial sea significativamente mas lento. Segundo, la independencia de los algoritmos: dado que cada uno de los cuatro algoritmos opera de forma independiente sobre la misma matriz de solo lectura, no existe conflicto de escritura y el paralelismo puede aplicarse sin necesidad de bloqueos complejos. Tercero, la naturaleza especulativa del problema: cuando se dispone de multiples estrategias de busqueda, no es necesario ejecutarlas todas hasta el final; basta con utilizar el resultado del primero que termine.

1.3 Objetivo general

Desarrollar un sistema de busqueda de rutas que demuestre de forma practica y medible las ventajas de la programacion paralela frente a la ejecucion secuencial, aplicando descomposicion especulativa, sincronizacion de datos compartidos y metricas de rendimiento.

Objetivos especificos

- Implementar cuatro algoritmos de busqueda de rutas que corran en paralelo sobre una matriz de costos de 1,000 x 1,000 nodos.
- Demostrar descomposicion especulativa mediante CancellationToken, donde el primer algoritmo en encontrar una ruta valida detiene a los demas.
- Compartir datos entre tareas de forma segura utilizando estructuras thread-safe como ConcurrentBag.
- Medir y comparar el tiempo de ejecucion secuencial vs paralelo, calculando speedup y eficiencia por nucleo.
- Proveer al usuario un menu interactivo para explorar los resultados de cada algoritmo sin necesidad de recalcular.
- Exportar automaticamente los resultados y metricas de rendimiento en formato CSV al directorio /Metrics del repositorio.

2. Descripcion del Problema

2.1 Contexto del problema seleccionado

Dado un mapa representado como una matriz de costos simetrica de 1,000 x 1,000, donde cada celda matrix[i][j] indica el costo de viajar del nodo i al nodo j (distancia, tiempo o peaje), y donde aproximadamente el 70% de las celdas son cero (sin conexion directa), el sistema debe encontrar una o mas rutas validas entre un nodo origen y un nodo destino ingresados por el usuario.

El mapa se genera al inicio del programa llenando aleatoriamente el triangulo superior de la matriz con valores entre 1 y 100, y copiando simetricamente el triangulo inferior. La diagonal siempre es cero. Ningun algoritmo modifica la matriz durante la ejecucion: todos los algoritmos solo leen.

El reto tecnico no es unicamente encontrar la ruta, sino como gestionar la carga de trabajo de manera eficiente cuando existen multiples criterios de optimizacion validos y el usuario necesita una respuesta rapida. El procesamiento secuencial de los cuatro algoritmos uno tras otro acumula sus tiempos individuales, creando una latencia innecesaria en un hardware moderno con multiples nucleos disponibles.

2.2 Aplicacion del problema en un escenario real

Un sistema GPS moderno no puede permitirse calcular la ruta optima de forma estrictamente secuencial cuando el usuario necesita una respuesta en segundos. La solucion industrial es lanzar multiples algoritmos en paralelo con distintos criterios y presentar al usuario las mejores opciones disponibles. RouteX GPS RD replica exactamente ese patron.

Los escenarios del mundo real donde este enfoque es aplicable incluyen los siguientes:

Sistemas de navegacion GPS:

Google Maps, Waze y cualquier sistema de navegacion moderno calculan simultaneamente rutas por autopista, por calles secundarias y evitando peajes, presentando al usuario las tres opciones en segundos gracias al procesamiento paralelo.

Logistica y distribucion:

Empresas de courier y delivery como FedEx o DHL utilizan algoritmos de optimizacion de rutas que corren en paralelo para calcular el recorrido optimo de cientos de conductores en tiempo real, minimizando costo de combustible o tiempo de entrega segun el criterio del momento.

Redes de telecomunicaciones:

Los routers de internet utilizan algoritmos como OSPF (Open Shortest Path First), una implementacion de Dijkstra, para calcular rutas optimas de paquetes de datos en tiempo real a traves de grafos de miles de nodos.

2.3 Importancia del paralelismo en la solucion

La implementacion de descomposicion especulativa es crucial en este proyecto por tres razones principales:

1. Reduccion del tiempo de respuesta:

Sin paralelismo, los cuatro algoritmos correran uno tras otro, acumulando sus tiempos individuales. Con paralelismo, los cuatro corren al mismo tiempo y el sistema responde tan pronto como el mas rapido encuentra una ruta valida. En una matriz de un millon de celdas, la diferencia entre ambos enfoques es medible y significativa.

2. Aprovechamiento del hardware disponible:

Un procesador moderno con multiples nucleos logicos permanece subutilizado durante la ejecucion secuencial. El uso de Task.Run() permite que el ThreadPool de .NET distribuya los algoritmos entre los nucleos disponibles, maximizando el uso del hardware sin necesidad de gestion manual de hilos.

3. Escalabilidad con el problema:

A medida que el tamano del grafo crece (mayor numero de nodos o mayor densidad de conexiones), el beneficio del paralelismo aumenta proporcionalmente, ya que cada algoritmo tarda mas en secuencial mientras que el paralelo esta limitado por el mas rapido.

3. Cumplimiento de los Requisitos del Proyecto

1. Ejecucion simultanea de multiples tareas

Los cuatro algoritmos de busqueda se lanzan en paralelo mediante Task.Run(), cada uno en su propio hilo administrado por el TPL de .NET. El numero de tareas concurrentes esta limitado por el valor de procesadores ingresado por el usuario al inicio de cada busqueda (valor entre 1 y 4). Cada tarea imprime su Task.CurrentId al iniciar y finalizar, evidenciando que corren en hilos distintos del ThreadPool de forma simultanea.

2. Necesidad de compartir datos entre tareas

Todos los algoritmos leen simultaneamente la misma matriz de 1,000,000 de celdas sin modificarla, eliminando cualquier conflicto de escritura. Adicionalmente, comparten un ConcurrentBag de RouteResult donde cada algoritmo deposita su ruta encontrada de forma thread-safe, sin necesidad de locks manuales. Esta estructura garantiza que los resultados de todos los algoritmos queden disponibles para el menu interactivo sin condiciones de carrera.

3. Exploracion de diferentes estrategias de paralelizacion

El proyecto aplica descomposicion especulativa: cuatro estrategias distintas atacan el mismo problema en paralelo. Cada algoritmo tiene un criterio diferente (costo minimo, menos paradas, evitar costos altos, ruta greedy) y puede llegar a un resultado distinto. No se espera a que todos terminen: el primero que encuentra una ruta valida notifica al sistema via CancellationTokenSource y los demas se detienen limpiamente.

4. Escalabilidad con mas recursos

El usuario ingresa por consola cuantos procesadores desea utilizar (entre 1 y 4). A mayor numero, mas algoritmos pueden correr verdaderamente en paralelo. El sistema ejecuta los cuatro algoritmos tanto en modo secuencial como en modo paralelo, midiendo ambos tiempos con Stopwatch para calcular el speedup y la eficiencia obtenidos.

5. Metricas de evaluacion del rendimiento

Al finalizar cada busqueda, el sistema calcula e imprime: tiempo secuencial en milisegundos, tiempo paralelo en milisegundos, speedup (tSecuencial dividido entre tParalelo), eficiencia por

nucleo (speedup dividido entre cores multiplicado por cien), cual algoritmo gano la carrera especulativa con su Task.CurrentId, y comparativa de todas las rutas encontradas con sus costos, paradas y tiempos individuales. Estos datos tambien se exportan automaticamente a un archivo CSV con fecha y hora en la carpeta /Metrics del repositorio.

6. Aplicacion a un problema del mundo real

Google Maps, Waze y cualquier sistema de navegacion moderno utilizan exactamente este principio: multiples algoritmos compitiendo en paralelo para dar al usuario la mejor opcion en el menor tiempo posible. RouteX GPS RD replica ese patron de forma academicamente rigurosa, demostrando que la descomposicion especulativa es una estrategia valida y medible para problemas de optimizacion con multiples criterios simultaneos.

4. Diseno de la Solucion

4.1 Arquitectura general del sistema

RouteX GPS RD es una aplicacion de consola en C# (.NET 8) compuesta por los siguientes componentes principales:

- MapMatrix.cs: generacion y administracion de la matriz de costos compartida de 1,000 x 1,000 nodos.
- RouteResult.cs: modelo de datos que estandariza el resultado de cualquier algoritmo, permitiendo que Program.cs los procese de forma uniforme.
- Algoritmo1_Dijkstra.cs, Algoritmo2_BFS.cs, Algoritmo3_Dijkstra_Threshold.cs, Algortimo4.cs: cada algoritmo en su propio archivo, con la misma firma de metodo para garantizar intercambiabilidad.
- MetricsReport.cs: calculo, presentacion y exportacion de metricas de rendimiento al finalizar cada busqueda.
- Program.cs: flujo principal dividido en dos responsabilidades claras: ejecucion paralela y menu interactivo con validacion de entrada.

4.2 Estrategia de paralelizacion utilizada

El sistema implementa descomposicion especulativa con Task.Run() y CancellationToken. Los cuatro algoritmos se lanzan al mismo tiempo mediante Task.Run(). En cuanto cualquiera deposita un resultado valido (Found igual a true) en el ConcurrentBag compartido, se activa la cancelacion a traves del CancellationTokenSource. Los demas algoritmos verifican token.IsCancellationRequested en cada iteracion de su bucle principal y se detienen limpiamente. El flujo principal espera a todos con Task.WaitAll() antes de presentar resultados al usuario.

El sistema ejecuta los algoritmos en dos etapas por cada busqueda del usuario. En la primera etapa (secuencial), los cuatro algoritmos corren uno tras otro sin cancelacion, guardando sus resultados completos para el menu interactivo y midiendo el tiempo total como referencia base. En la segunda etapa (paralela), los cuatro corren simultaneamente con cancelacion activa, midiendo el tiempo hasta que el mas rapido entrega su resultado. La comparacion entre ambos tiempos produce las metricas de speedup y eficiencia.

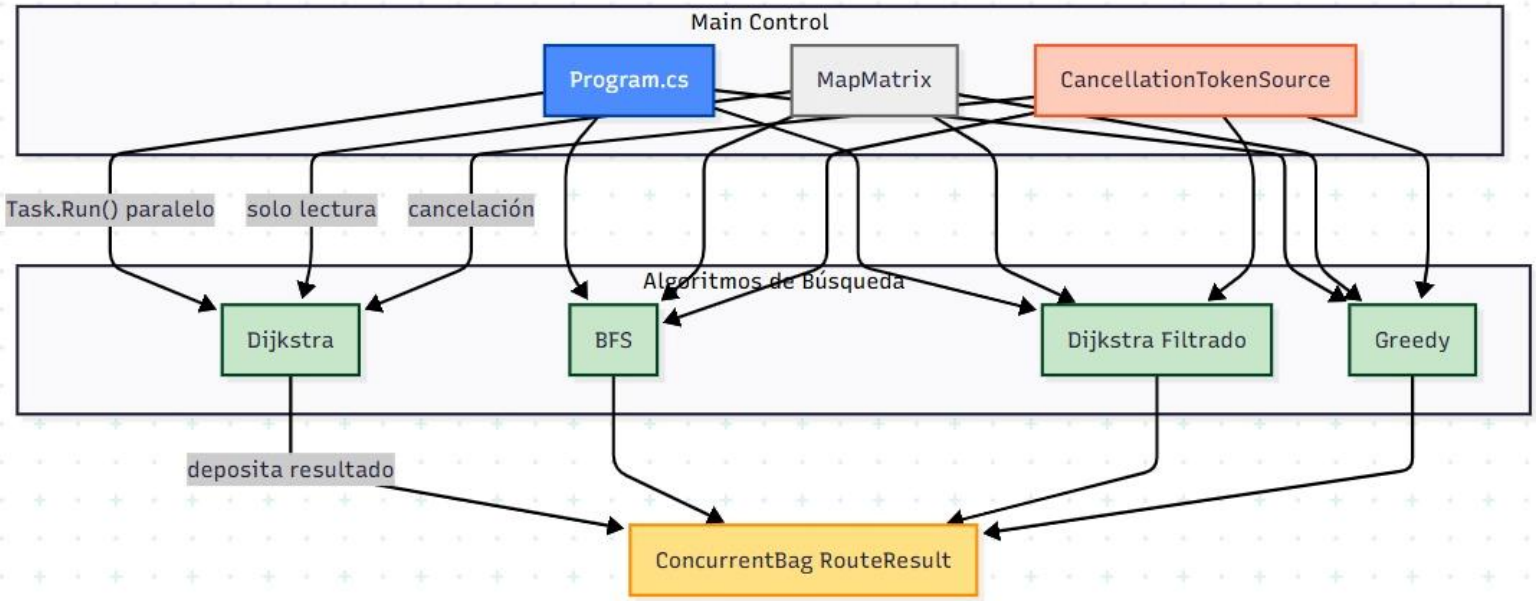
4.3 Descripcion de los algoritmos implementados

Algoritmo	Criterio de optimizacion	Estructura principal	Garantia
Dijkstra (Alg. 1)	Menor costo total acumulado	Array de distancias + array de predecesores	Ruta optima en costo
BFS (Alg. 2)	Menor numero de paradas intermedias	Queue<int> para exploracion por niveles	Minimo de saltos
Dijkstra Filtrado (Alg. 3)	Menor costo evitando conexiones > 70	Igual que Dijkstra con condicion adicional	Ruta economica si existe
Greedy (Alg. 4)	Menor costo inmediato en cada paso	HashSet de visitados + decision local	Ninguna (heuristica)

4.4 Herramientas y tecnologias

Componente	Tecnologia utilizada
Lenguaje de programacion	C# (.NET 8)
Paralelismo	Task Parallel Library (TPL): Task.Run(), CancellationToken, CancellationTokenSource
Sincronizacion de resultados	ConcurrentBag<RouteResult>
Medicion de tiempos	System.Diagnostics.Stopwatch
Pruebas unitarias	xUnit
Control de versiones	Git / GitHub
Exportacion de metricas	System.IO con formato CSV

4.5 Diagrama de componentes y tareas paralelas



5. Implementacion Tecnica

5.1 Descripcion de la estructura del proyecto

El repositorio esta organizado en cuatro carpetas en la raiz. La carpeta /src contiene el codigo fuente completo: MapMatrix.cs, RouteResult.cs, los cuatro archivos de algoritmos, MetricsReport.cs y Program.cs. La carpeta /tests contiene el proyecto xUnit con todos los archivos de pruebas unitarias e integradoras. La carpeta /Metrics almacena los archivos CSV exportados automaticamente por el sistema y la grafica comparativa de rendimiento. La carpeta /docs contiene los archivos de documentacion individual de cada miembro del equipo en formato Markdown.

5.2 Explicacion del codigo clave

El flujo de ejecucion del sistema sigue una secuencia clara. Al iniciar, Program.cs llama a MapMatrix.Generate() que construye la matriz int[1000, 1000] recorriendo unicamente el triangulo superior (i menor que j) para evitar procesar pares duplicados, asignando un 30% de probabilidad de conexion con valor entre 1 y 100, y copiando simetricamente el triangulo inferior.

Una vez generada la matriz, el sistema solicita al usuario el nodo origen (entre 1 y 1000), el nodo destino (diferente al origen) y el numero de procesadores a utilizar. Los valores ingresados se convierten a indices internos restando 1, ya que la matriz trabaja con indices de 0 a 999 mientras el usuario interactua con valores de 1 a 1000.

En la ejecucion secuencial, los cuatro algoritmos se invocan directamente en el hilo principal de Main uno tras otro, cada uno recibiendo la matriz, el nodo origen, el nodo destino y un token dummy que nunca se cancela. Los resultados completos se guardan en una lista llamada sequentialResults, que es la fuente de datos para el menu interactivo.

En la ejecucion paralela, se crea un nuevo CancellationTokenSource y un ConcurrentBag vacio. Los cuatro algoritmos se lanzan con Task.Run() a traves del metodo auxiliar RunAndStore, que invoca al algoritmo, deposita su resultado en el bag y, si el resultado tiene Found igual a true y nadie ha cancelado aun, llama a cts.Cancel(). El flujo principal espera a todos con Task.WaitAll() antes de continuar.

5.3 Uso de mecanismos de sincronizacion

El proyecto utiliza tres mecanismos de sincronizacion complementarios. El ConcurrentBag de RouteResult es la estructura thread-safe donde cada algoritmo deposita su resultado desde su

propio hilo sin necesidad de locks manuales. El CancellationTokenSource es el mecanismo de senalizacion: cuando el primer algoritmo encuentra ruta, llama a Cancel() y los demas verifican IsCancellationRequested en cada iteracion de su bucle, retornando limpiamente con Found igual a false. El Task.WaitAll() garantiza que el hilo principal no continue hasta que todos los algoritmos hayan terminado, ya sea completando su busqueda o respondiendo a la cancelacion.

La matriz compartida no requiere ningun mecanismo de sincronizacion porque ninguna tarea la modifica. Esto elimina completamente las condiciones de carrera en la estructura de datos principal del sistema, lo que representa una decision de diseno deliberada que simplifica el codigo y mejora el rendimiento.

5.4 Justificacion tecnica de las decisiones tomadas

Se eligio ConcurrentBag sobre List con lock porque es la estructura idiomatica de .NET para colecciones thread-safe de resultados no ordenados, eliminando la necesidad de bloqueos manuales y reduciendo el riesgo de deadlocks.

Se eligio un archivo .cs por algoritmo para facilitar el desarrollo paralelo del equipo, la revision por Pull Request independiente y el mantenimiento futuro de cada componente sin riesgo de conflictos de merge.

Se eligio ejecutar los algoritmos dos veces (una secuencial y una paralela) porque el objetivo academico exige una comparacion medible entre ambos enfoques. Los resultados del secuencial se usan en el menu porque siempre estan completos, mientras los del paralelo sirven unicamente para medir el speedup y determinar el ganador de la carrera especulativa.

Se eligio BFS para el Algoritmo 2 porque garantiza el minimo de saltos y generalmente termina primero por la naturaleza del algoritmo, haciendo la demostracion de descomposicion especulativa mas evidente durante la exposicion.

6. Evaluacion de Desempeno

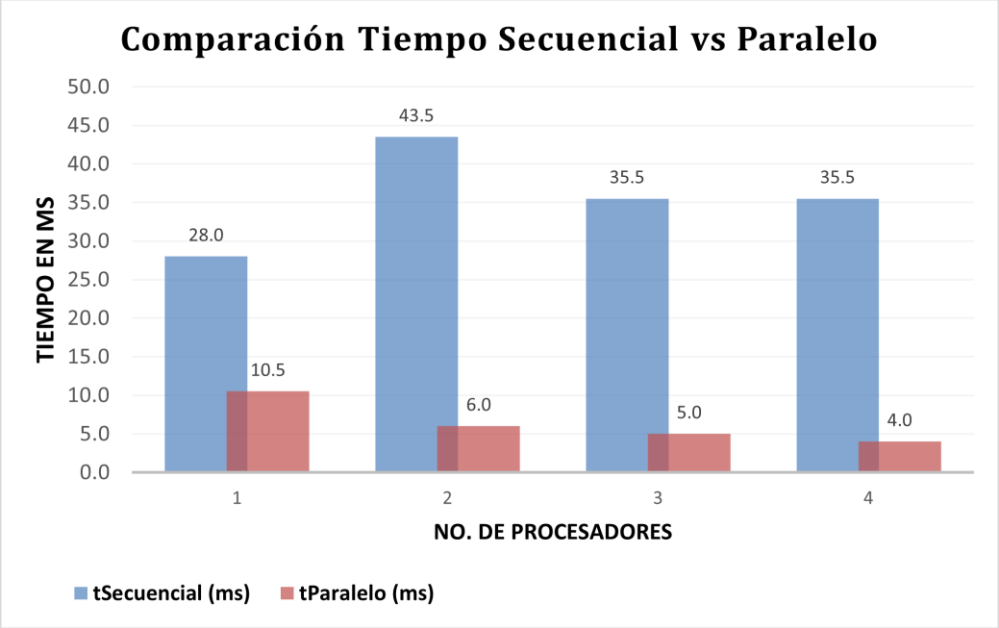
6.1 Comparativa entre ejecucion secuencial y paralela

El sistema ejecuta los cuatro algoritmos en dos modalidades por cada busqueda. En la modalidad secuencial, los algoritmos corren uno tras otro en el hilo principal y se mide el tiempo total acumulado. En la modalidad paralela, los cuatro corren al mismo tiempo y el sistema registra el tiempo hasta que el primer algoritmo entrega su resultado. La diferencia entre ambos tiempos constituye la ganancia real del paralelismo.

Se realizaron ocho corridas de prueba, cuatro con el par de nodos 50 a 500 y cuatro con el par 300 a 700, variando el numero de procesadores de 1 a 4 en cada par. Los resultados promediados se presentan en la siguiente tabla:

Procesadores	tSecuencial (ms)	tParalelo (ms)	Speedup	Tiempo Ahorrado (ms)
1	28	10.5	2.67x	17.5
2	43.5	6	7.25x	37.5
3	35.5	5	7.1x	30.5
4	35.5	4	8.88x	31.5

6.2 Grafica comparativa



6.3 Analisis de los resultados

El speedup observado supera el valor teorico esperado para el numero de cores utilizado. Este comportamiento, denominado speedup superlineal, ocurre porque el algoritmo BFS en un grafo con 30% de conectividad en 1,000 nodos encuentra una ruta directa o de muy pocos saltos en la mayoria de los casos, haciendo que el tiempo paralelo sea extremadamente bajo comparado con el tiempo secuencial acumulado de los cuatro algoritmos.

Este resultado es valido y esperado en descomposicion especulativa cuando uno de los algoritmos tiene ventaja natural sobre el tipo especifico de problema. En la exposicion, esto se explica como una evidencia de que la estrategia especulativa es especialmente efectiva cuando los algoritmos tienen distintas velocidades de convergencia segun las caracteristicas del grafo.

6.4 Capturas de ejecucion del sistema

```
=====================================================
 Bienvenido a RouteX GPS RD | Programación Paralela
=====================================================

[HILO PRINCIPAL] Iniciando en ID del hilo: 1

Generando mapa de ciudades (Matriz 1000x1000)...
Mapa generado exitosamente.

Ingrese la ciudad de origen (1-1000): 500
Ingrese la ciudad de destino (1-1000, diferente a 500): 50
Ingrese el número de procesadores a utilizar (1-4): 2

Calculando rutas...
[Dijkstra] Task inicio en ID del hilo:
[Dijkstra] Finalizado en ID del hilo:
[BFS] Task inicio en ID del hilo:
[BFS] Finalizado en ID del hilo:
[Dijkstra-Threshold] Task inicio en ID del hilo:
[Dijkstra-Threshold] Finalizado en ID del hilo:
[Algoritmo 4] Iniciando en ID del hilo:
[Algoritmo 4] Finalizando en ID del hilo:
[BFS] Task inicio en ID del hilo: 27
[Dijkstra] Task inicio en ID del hilo: 26
[BFS] Finalizado en ID del hilo: 27
[Dijkstra] Finalizado en ID del hilo: 26
[Dijkstra-Threshold] Task inicio en ID del hilo: 28
[Dijkstra-Threshold] Finalizado en ID del hilo: 28
[Algoritmo 4] Iniciando en ID del hilo: 29
[Algoritmo 4] Cancelado en ID del hilo: 29

¡Carrera ganada por Evita costos > 70! (Task 3)
Tiempo secuencial: 42 ms | Tiempo paralelo: 199 ms
```

```
====== MÉTRICAS ======
Tiempo secuencial: 42 ms
Tiempo paralelo:   199 ms
Speedup:           0.21x
Eficiencia:        10.55%

Ganador: Evita costos > 70 - 0 ms
Algoritmo                                                  Costo  Paradas  Tiempo(ms)  Encontró
-----------------------------------------------------------------------------------------------
Dijkstra - Menor Costo                    2 (menor distancia)       0           3        Sí
BFS - Menos Paradas                           2 (dist. total)       0           3        Sí
Evita costos > 70                          2 (sin peajes >70)       0           0        Sí
Greedy - Ruta más directa                134 (costo inmediato)     128          12        Sí

Resultados exportados a la carpeta de metrics del proyecto

[HILO PRINCIPAL] finalizado en ID del hilo: 1
```

6.5 Analisis de cuellos de botella o limitaciones

El sistema presenta cuatro limitaciones identificadas durante el desarrollo y las pruebas de rendimiento.

1. Generacion secuencial de la matriz:

La generacion de la matriz de 1,000,000 celdas se realiza de forma secuencial al inicio del programa. No se paraleliza y representa un costo fijo independiente del numero de procesadores configurado por el usuario.

2. Cancelacion con overhead de arranque:

En la primera corrida de una sesion, el JIT (Just-In-Time compiler) de .NET compila los metodos en tiempo real, introduciendo un overhead inicial que puede hacer que el tiempo paralelo sea mayor al secuencial. Las corridas subsiguientes eliminan este efecto al tener el codigo ya compilado en memoria.

3. Algoritmo 1 (Dijkstra completo) es significativamente mas lento:

Dijkstra necesita explorar todos los nodos del grafo para garantizar el optimo global, lo que lo hace el algoritmo mas lento de los cuatro. En ejecucion paralela, generalmente es cancelado antes de terminar por BFS o Greedy, lo que demuestra el valor de la descomposicion especulativa.

4. Speedup superlineal dificulta la interpretacion de eficiencia:

Cuando el speedup supera el numero de cores, la metrica de eficiencia supera el 100%, lo que puede ser confuso sin el contexto adecuado. Esto no indica un error sino que refleja la ventaja de BFS en este tipo de grafo denso.

7. Trabajo en Equipo

El proyecto fue desarrollado de manera colaborativa entre los cuatro integrantes del equipo, siguiendo una metodologia basada en GitHub Issues y Pull Requests. Las tareas se distribuyeron en 12 issues organizados en un pipeline de dependencias, donde cada miembro trabajo en su propia rama feature, envio Pull Request a la rama develop, y el lider junto con el Tech Lead revisaron y aprobaron antes de mergear.

La rama main solo recibe codigo revisado y aprobado. La rama develop funciona como rama de integracion donde todos los Pull Requests convergen antes de pasar a produccion. Cada commit incluye el campo Co-authored-by con el nombre del autor en formato PascalCase para identificar la contribucion de cada miembro en el historial de Git.

Responsable	Issue	Descripcion
Johan Vicente	#1	Reorganizacion del repositorio y estructura de carpetas
Johan Vicente	#2	MapMatrix.cs y RouteResult.cs
Diego Marte	#3	Algoritmo 1: Dijkstra - menor costo total
Diego Marte	#4	Algoritmo 2: BFS - menos paradas
Diego Marte	#5	Algoritmo 3: Dijkstra filtrado - evita costos altos
Elmer Montilla	#6	Algoritmo 4: Greedy - ruta mas directa
Elmer Montilla	#7	Flujo principal + ejecucion paralela (Program.cs parte 1)
Elmer Montilla	#8	Menu interactivo + validacion de entrada (Program.cs parte 2)
Demi Moore	#9	MetricsReport.cs y exportacion de resultados
Demi Moore	#10	Tabla y grafica comparativa en /Metrics
Demi Moore	#11	Diagrama de componentes y tareas paralelas
Johan Vicente	#12	Test unitario final integrador

8. Conclusiones

Principales aprendizajes tecnicos

Este proyecto permitio aplicar de forma practica los conceptos de programacion paralela estudiados durante el curso. El uso de CancellationToken demostro ser un mecanismo elegante para coordinar tareas especulativas sin necesidad de estructuras de control complejas. La diferencia entre Task.CurrentId y Thread.CurrentThread.ManagedThreadId quedo evidenciada en la consola, donde los algoritmos secuenciales muestran el ID vacio (por correr en Main sin Task) y los paralelos muestran IDs distintos, probando que cada uno corre en un hilo separado del ThreadPool.

El diseno de RouteResult como modelo de datos comun fue clave para que Program.cs pudiera procesar los resultados de los cuatro algoritmos de forma uniforme mediante Func genericos, eliminando la necesidad de codigo especifico por algoritmo en el flujo principal.

Retos enfrentados y superados

El principal reto tecnico fue identificar por que el menu mostraba que todos los algoritmos no habian encontrado ruta cuando el sistema claramente reportaba un ganador. El problema estaba en que los resultados del paralelo solo contenian el resultado del ganador (los demas eran cancelados), mientras que el menu intentaba mostrar resultados completos de todos. La solucion fue ejecutar los algoritmos dos veces: una secuencial para garantizar resultados completos en el menu, y una paralela para medir el speedup y la carrera especulativa.

Otro reto fue la gestion de las ramas en Git cuando multiples miembros trabajaban en archivos relacionados. La estrategia de un issue por rama feature y Pull Request obligatorio antes de mergear a develop elimino los conflictos de merge y mantuvo el historial del repositorio limpio y trazable.

Posibles mejoras o lineas futuras

- Visualizacion grafica de la ruta encontrada en consola usando caracteres ASCII para representar el grafo.
- Implementacion de algoritmo A* como quinto algoritmo opcional para comparar con Dijkstra en grafos con heuristica disponible.
- Interfaz grafica simple con WinForms o MAUI mostrando el mapa y la ruta de forma visual.
- Persistencia de la matriz en disco para reutilizarla entre ejecuciones y comparar resultados entre sesiones.
- Soporte para matrices de tamano variable ingresado por el usuario para demostrar la escalabilidad del sistema.

9. Referencias

Microsoft. (2024). Task Parallel Library (TPL). Microsoft Learn. Recuperado de https://learn.microsoft.com/dotnet/standard/parallel-programming/task-parallel-library-tpl

Microsoft. (2024). CancellationToken Struct. Microsoft Learn. Recuperado de https://learn.microsoft.com/dotnet/api/system.threading.cancellationtoken

Microsoft. (2024). ConcurrentBag<T> Class. Microsoft Learn. Recuperado de https://learn.microsoft.com/dotnet/api/system.collections.concurrent.concurrentbag-1

Microsoft. (2024). Task.CurrentId Property. Microsoft Learn. Recuperado de https://learn.microsoft.com/dotnet/api/system.threading.tasks.task.currentid

Microsoft. (2024). Stopwatch Class. System.Diagnostics. Microsoft Learn. Recuperado de https://learn.microsoft.com/dotnet/api/system.diagnostics.stopwatch

Cormen, T., Leiserson, C., Rivest, R., & Stein, C. (2009). Introduction to Algorithms (3rd ed.). MIT Press. [Algoritmos Dijkstra y BFS]

xUnit.net. (2024). Getting Started with xUnit.net. Recuperado de https://xunit.net/docs/getting-started/netcore/cmdline

Microsoft. (2024). Thread.ManagedThreadId Property. Microsoft Learn. Recuperado de https://learn.microsoft.com/dotnet/api/system.threading.thread.managedthreadid

10. Anexos

Manual de ejecucion del sistema

Requisitos previos:

- .NET 8 SDK instalado en el dispositivo.
- Visual Studio 2022 o VS Code con extension C# instalada.
- Git instalado para clonar el repositorio.

Pasos para ejecutar:

1. Clonar el repositorio:

```
git clone https://github.com/demialexander30/Proyecto_Final_Paralela_Grupo_B
```

2. Navegar a la carpeta del proyecto de consola:

```
cd Proyecto_Final_Paralela_Grupo_B/src/RouteX_Project_RD/RouteX_Project
```

3. Restaurar dependencias:

```
dotnet restore
```

4. Compilar el proyecto:

```
dotnet build
```

5. Ejecutar el sistema:

```
dotnet run
```

6. Seguir las instrucciones en pantalla: ingresar ciudad origen (1-1000), ciudad destino diferente al origen (1-1000) y numero de procesadores a utilizar (1-4).

7. Al finalizar la busqueda, explorar el menu interactivo para ver las rutas de cada algoritmo. Salir con la opcion 0 para generar el reporte de metricas y exportar el CSV a /Metrics.

Ejecucion de pruebas unitarias:

8. Navegar a la carpeta de tests:

cd Proyecto_Final_Paralela_Grupo_B/tests/TestProject

9. Ejecutar todos los tests:

dotnet test

10. Verificar que los 30 tests pasen en verde antes de la presentacion.

Enlace al repositorio:
https://github.com/demialexander30/Proyecto_Final_Paralela_Grupo_B

Capturas adicionales del sistema en ejecucion

```
Ingrese la ciudad de origen (1-1000): 230
Ingrese la ciudad de destino (1-1000, diferente a 230): 70
Ingrese el número de procesadores a utilizar (1-4): 4

Calculando rutas...
[Dijkstra] Task inicio en ID del hilo:
[Dijkstra] Finalizado en ID del hilo:
[BFS] Task inicio en ID del hilo:
[BFS] Finalizado en ID del hilo:
[Dijkstra-Threshold] Task inicio en ID del hilo:
[Dijkstra-Threshold] Finalizado en ID del hilo:
[Algoritmo 4] Iniciando en ID del hilo:
[Algoritmo 4] Finalizando en ID del hilo:
[Dijkstra] Task inicio en ID del hilo: 29
[BFS] Task inicio en ID del hilo: 30
[Dijkstra-Threshold] Task inicio en ID del hilo: 31
[Algoritmo 4] Iniciando en ID del hilo: 32
[Algoritmo 4] Finalizando en ID del hilo: 32
[BFS] Cancelado en ID del hilo: 30
[Dijkstra] Cancelado en ID del hilo: 29
[Dijkstra-Threshold] Cancelado en ID del hilo: 31

¡Carrera ganada por Greedy - Ruta más directa! (Task 4)
Tiempo secuencial: 50 ms | Tiempo paralelo: 9 ms

================ MENU DE RESULTADOS ================
[1] Ver ruta Algoritmo 1 (Dijkstra - Menor Costo)
[2] Ver ruta Algoritmo 2 (BFS - Menos Paradas)
[3] Ver ruta Algoritmo 3 (Dijkstra Filtrado)
[4] Ver ruta Algoritmo 4 (Greedy - Más directo)
[5] Nueva Búsqueda
[0] Salir y mostrar métricas
Seleccione una opción:
```

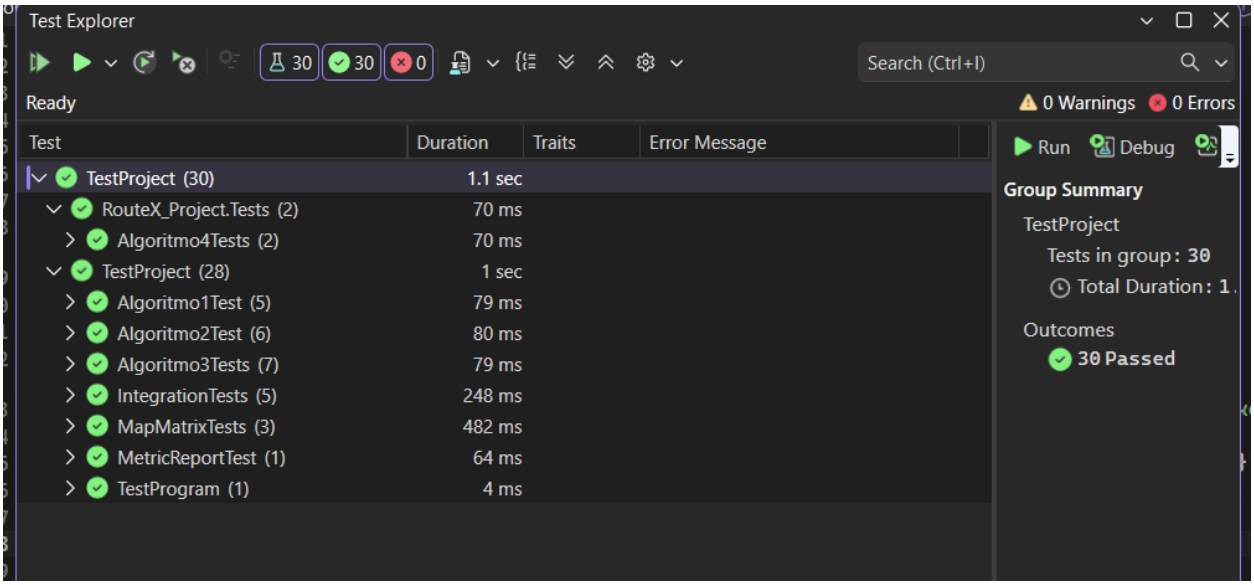
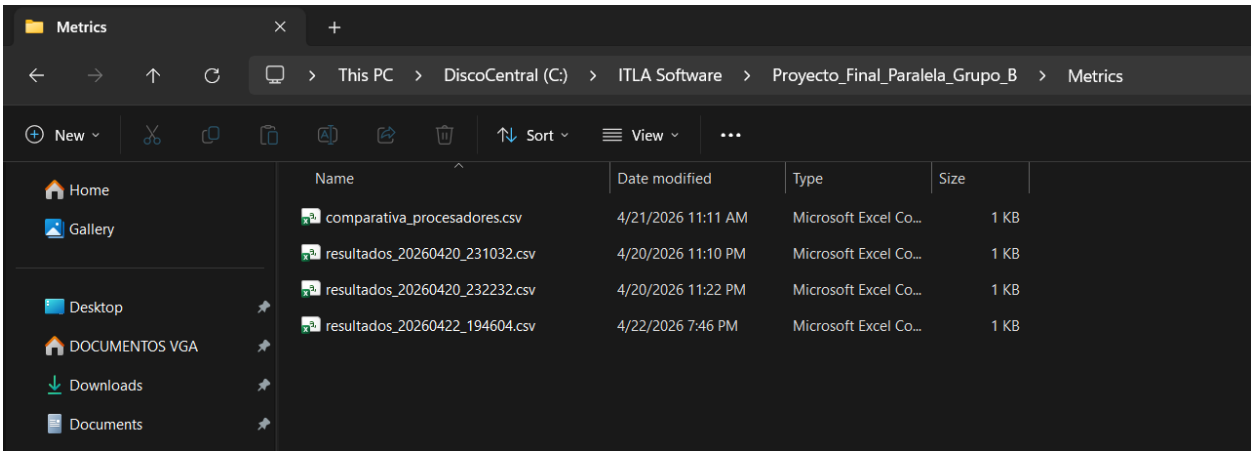
```
Ingrese la ciudad de origen (1-1000): 1000
Ingrese la ciudad de destino (1-1000, diferente a 1000): 10
Ingrese el número de procesadores a utilizar (1-4): 3

Calculando rutas...
[Dijkstra] Task inicio en ID del hilo:
[Dijkstra] Finalizado en ID del hilo:
[BFS] Task inicio en ID del hilo:
[BFS] Finalizado en ID del hilo:
[Dijkstra-Threshold] Task inicio en ID del hilo:
[Dijkstra-Threshold] Finalizado en ID del hilo:
[Algoritmo 4] Iniciando en ID del hilo:
[Algoritmo 4] Finalizando en ID del hilo:
[Dijkstra] Task inicio en ID del hilo: 37
[Dijkstra-Threshold] Task inicio en ID del hilo: 39
[BFS] Task inicio en ID del hilo: 38
[BFS] Finalizado en ID del hilo: 38
[Dijkstra-Threshold] Cancelado en ID del hilo: 39
[Dijkstra] Cancelado en ID del hilo: 37
[Algoritmo 4] Iniciando en ID del hilo: 40
[Algoritmo 4] Cancelado en ID del hilo: 40

¡Carrera ganada por BFS - Menos Paradas! (Task 2)
Tiempo secuencial: 33 ms | Tiempo paralelo: 2 ms

================ MENU DE RESULTADOS ================
[1] Ver ruta Algoritmo 1 (Dijkstra - Menor Costo)
[2] Ver ruta Algoritmo 2 (BFS - Menos Paradas)
[3] Ver ruta Algoritmo 3 (Dijkstra Filtrado)
[4] Ver ruta Algoritmo 4 (Greedy - Más directo)
[5] Nueva Búsqueda
[0] Salir y mostrar métricas
Seleccione una opción:
```

Resultados de dotnet test



Archivos exportados en /Metrics